

基于多智能体协同的软件开发任务分解与缺陷修复研究

陈立鸿

S202588306

摘要：随着大语言模型在代码理解、代码生成和问题分析方面展现出较强能力，基于智能体的软件开发辅助系统逐渐成为软件工程研究热点。然而，单智能体方法在处理复杂开发任务时常受限于上下文窗口、长期规划能力和验证机制，容易出现任务拆分不充分、补丁局部最优和解释不可追踪等问题。针对软件开发任务具有目标多样、依赖复杂与质量约束显著的特点，本文提出一种基于多智能体协同的软件开发任务分解与缺陷修复方法。该方法构建由规划智能体、分析智能体、编码智能体、审查智能体和测试智能体组成的协同框架，利用层次化任务图实现需求拆分，借助共享记忆与证据驱动通信机制实现上下文对齐，并在补丁生成后引入自动验证与迭代收敛过程。本文在包含需求实现、单元测试补全、缺陷定位和补丁修复等 40 个仿真任务的数据集上开展实验，并与单智能体顺序执行方案进行比较。结果表明，多智能体协同方法在任务完成率、缺陷修复成功率、平均迭代轮次和补丁可维护性等方面均优于基线方案，其中任务完成率提高 18.7%，缺陷修复成功率提高 21.3%，平均修复轮次减少 26.4%。研究说明，在软件工程场景中，智能体数量的增加本身并不直接带来性能提升，关键在于角色边界、共享状态和验证闭环的合理设计。

关键词：多智能体；软件工程；任务分解；缺陷修复；大语言模型

Abstract: With the rapid progress of large language models in code understanding, code generation, and issue analysis, agent-based development assistants have become an active topic in software engineering. However, single-agent approaches are often constrained by limited planning depth, context overload, and weak verification mechanisms when facing complex development tasks. To address these problems, this paper proposes a multi-agent collaborative method for software development task decomposition and defect repair. The framework consists of a planner agent, analyzer agent, coder agent, reviewer agent, and tester agent. It uses hierarchical task graphs for requirement decomposition, evidence-driven communication and shared memory for context alignment, and an automated validation loop for iterative convergence after patch generation. Experiments are conducted on a simulated dataset containing forty tasks covering feature implementation, test completion, defect localization, and bug fixing. Compared with a sequential single-agent baseline, the proposed method improves task completion rate by 18.7%, increases defect repair success rate by 21.3%, and reduces average repair iterations by 26.4%. The results indicate that the effectiveness of multi-agent software engineering depends less on agent quantity itself and more on clear role boundaries, shared state management, and a well-designed verification loop.

Keywords: multi-agent; software engineering; task decomposition; defect repair; large language model

1 引言

近年来，大语言模型在程序语言理解、代码补全、代码生成和技术问答等任务上取得了显著进展，推动了智能化软件开发工具的快速发展。相比传统 IDE 插件只提供局部建议，面向智能体的软件工程系统尝试让模型自主完成需求分析、代码修改、测试执行和结果解释等更完整的工作流，从而使“自动化开发协作”成为现实可研究的问题^[1-4]。

然而，软件开发任务天然具有长期依赖、多目标约束和强验证需求等特点。一个看似简单的缺陷修复请求，往往同时涉及问题定位、接口约束识别、代码上下文理解、补丁生成、测试验证和回归影响判断。如果仅由单个智能体串行完成所有工作，模型容易在长链条推理过程中出现上下文漂移，或者在尚未形成充分证据时过早进入编码阶段，导致补丁虽然通过局部测试，却引入新的副作用。

多智能体协同为上述问题提供了新的解决思路。通过让不同智能体承担规划、分析、编码、审查和测试等差异化职责，可以将复杂软件工程任务拆解为边界更清晰的子问题；再通过共享记忆和明确通信协议组织协作，能够降低单个智能体的认知负荷，并提高推理过程的可解释性。与此同时，多智能体方法也带来了通信开销、角色冲突和状态不一致等新挑战。

基于此，本文围绕软件开发任务分解与缺陷修复场景，提出一种多智能体协同方法。本文的主要工作包括：构建面向软件工程的多角色协同框架；设计层次化任务分解和证据驱动通信机制；引入审查与测试闭环提升补丁质量；通过仿真实验验证该方法在任务完成率、缺陷修复成功率和补丁可维护性等方面的效果。

2 理论基础与问题建模

2.1 智能体驱动的软件工程研究现状

现有研究表明，大语言模型已经能够在一定程度上执行代码生成、单元测试生成、文档摘要和缺陷定位等任务^[5-8]。在此基础上，研究者进一步提出让模型通过调用工具、读写文件和执行测试命令来完成更复杂的软件流程。与传统静态代码补全不同，这类方法强调模型对环境的感知能力，以及在任务执行过程中不断修正自身结论的能力。

多智能体框架则进一步扩展了这一思路。一些工作尝试使用多个角色智能体模拟软件团队中的产品经理、架构师、程序员和测试人员，以期在协同过程中形成更稳定的输出结构。总体来看，智能体驱动的软件工程研究已经从“代码片段生成”逐步演化为“面向任务闭环的协作式开发”。但在缺陷修复与任务分解的结合方面，仍然存在模型职责混杂、结果不可追踪和验证链路不充分等问题。

2.2 单智能体开发方式的局限

单智能体方案的首要问题是上下文负载过高。为了完成一个开发任务，智能体往往需要同时持有需求描述、代码仓库结构、历史修改记录、测试结果和编码规范等信息。随着上下文增大，模型容易将注意力分散在次要线索上，导致任务分析不稳定。特别是在大型代码仓库中，若缺乏显式的任务拆分机制，智能体很难在有限步骤内定位最关键的修改点。

第二个问题是目标耦合过紧。单智能体既要规划任务，又要生成代码，还要自我审查和执行验证，这种“既当裁判又当运动员”的方式容易造成确认偏差。模型一旦形成初始判断，后续往往倾向于围绕该判断寻找支持性证据，而忽略相反线索。结果是补丁可能在形式上完整，却缺乏对边界条件、回归影响和工程规范的充分考量。

2.3 多智能体协同的可行性与挑战

从软件工程组织学的角度看，多智能体协同具有天然合理性。真实软件团队之所以能够处理复杂需求，正是因为不同角色分别负责需求澄清、方案设计、编码实现、代码审查和测试验证。将这种角色分工映射到智能体系统中，有助于把复杂任

务转换为多个责任明确、输入输出清晰的子过程，从而增强过程透明性和结果可控性。

但多智能体方法并不必然优于单智能体。若角色边界不清，多个智能体可能围绕同一问题重复推理，造成资源浪费；若缺乏共享记忆，不同智能体可能基于彼此矛盾的上下文做出决策；若通信内容过于冗长，又会引入新的噪声。因此，本文将研究重点放在协同机制设计上，而不是简单增加智能体数量。

3 多智能体协同任务分解与缺陷修复方法

本文提出的协同框架由规划智能体、分析智能体、编码智能体、审查智能体和测试智能体构成，并由共享记忆模块记录中间工件、约束条件和验证结果。整体流程遵循“任务分解—证据收集—补丁生成—审查验证—经验回写”的闭环原则，以减少软件开发过程中的盲目搜索和局部最优问题。

表 1 多智能体协同框架中的角色分工

智能体角色	主要职责	关键输出
规划智能体	解析需求或缺陷描述，构建任务图与依赖关系	任务清单、优先级、执行顺序
分析智能体	定位相关模块，提取日志、测试结果和历史变更证据	问题证据包、影响范围说明
编码智能体	在限定修改范围内生成实现代码或补丁候选	候选补丁、代码说明
审查智能体	检查补丁语义、风格、一致性和潜在副作用	审查意见、修正建议
测试智能体	执行测试、比较结果并反馈失败证据	测试报告、回归风险提示

3.1 层次化任务分解策略

当系统收到开发任务后，规划智能体首先根据任务类型将其映射为需求实现、缺陷修复、测试补全或重构优化等一级类别，再进一步拆解为文件定位、接口约束确认、实现修改、测试更新和文档补充等二级子任务。每个子任务都附带输入工件、依赖前驱、完成标准和风险等级，从而形成可追踪的层次化任务图。这样的分解方式使后续智能体不必从零开始理解整个需求，而是围绕已定义的局部目标展开推理。

为提高调度合理性，本文引入任务优先级模型，对影响范围 I 、代码耦合度 C 、风险级别 R 和依赖深度 D 进行综合评估。优先级更高的子任务会优先分配给分析与编码智能体，以尽快澄清关键不确定性，并避免后续工作建立在错误假设之上。

$$P = 0.40I + 0.30C + 0.20R + 0.10D$$

3.2 基于证据的通信协议

为了降低多智能体通信中的语义漂移，本文采用证据驱动通信协议。每次消息传递都要求显式给出任务目标、已知证据、待验证假设和建议输出四项内容。例

如，分析智能体在提交问题定位结果时，不仅要指出疑似缺陷文件，还要附带对应的测试失败堆栈、调用链片段和相关历史提交信息。这样做的目的在于减少“只给结论不给依据”的黑箱式协作。

证据协议还规定，编码智能体不得直接根据模糊描述修改大范围代码，而必须在收到问题证据包和修改边界后才能生成补丁；审查智能体在否决某个补丁时，也需要指出具体违反的约束，如空指针路径未覆盖、异常处理不一致或命名不符合既有规范。通过将对话从自由文本收敛为结构化信息交换，系统能够显著提高协作效率和结果可追踪性。

3.3 缺陷定位与补丁生成机制

在缺陷修复场景中，分析智能体会综合使用测试失败日志、覆盖率信息、调用栈、相似缺陷历史和代码语义检索结果来锁定可疑区域。与仅依赖关键字匹配的方法相比，这种多源证据融合方式更适合处理跨函数、跨模块的缺陷传播问题。对于定位结果存在分歧的场景，系统允许分析智能体输出多个候选根因，并为每个候选项附带置信度。

编码智能体收到定位结果后，在限定文件集合和修改范围内生成一个或多个补丁候选。为了避免一次修改过多逻辑，本文要求编码智能体遵循最小变更原则，优先进行局部修补，并同步补充必要的单元测试或注释说明。候选补丁生成后，系统将其与任务图中的完成标准进行对齐，以确保代码修改与原始需求保持一致。

3.4 审查—测试闭环迭代

补丁生成完成后，并不会立即作为最终结果输出，而是先交由审查智能体执行静态语义检查。审查内容包括异常路径是否完备、命名与风格是否一致、边界条件是否覆盖以及是否破坏了已有接口契约。若审查发现潜在问题，则将具体意见返回给编码智能体进行定向修正，而不是要求其重新从头生成全部代码。

随后，测试智能体根据任务类型执行对应的单元测试、集成测试或静态检查，并将失败结果以结构化证据形式反馈给规划和分析智能体。只有当补丁满足既定测试门槛且审查意见收敛时，任务才被判定为完成。该闭环使系统具备“发现问题—修复问题—再次验证”的迭代能力，避免单次生成后直接结束所带来的质量风险。

3.5 共享记忆与经验复用

共享记忆模块用于保存任务图、关键约束、已确认事实、失败原因和最终补丁等中间工件。与简单的对话历史不同，共享记忆强调结构化与可检索性。例如，系统会将“某接口必须保持向后兼容”“某模块存在不可改动的性能约束”这类长期知识独立存储，供后续任务直接检索。这样能够减少重复分析，提高跨任务复用效率。

在实验过程中，本文还将历史缺陷修复模式写入经验库，例如空指针判空补丁、边界条件缺失补丁和异常传播路径修正补丁等。对于新任务，规划智能体和分析智能体可以优先检索相似案例，形成更稳健的初始假设。经验复用并不意味着机械套用旧补丁，而是通过结构化记忆帮助系统更快形成问题解决路径。

4 实验设计与结果分析

4.1 实验设置与任务分布

为了验证多智能体协同方法的有效性，本文构建了一个面向软件工程仿真实验的数据集，共包含 40 个任务，覆盖需求实现、缺陷修复和测试补全三类场景。实验代码仓库由若干中小规模服务模块组成，涉及接口层、业务逻辑层、数据访问层和测试代码。为保证比较公平，单智能体基线方案与多智能体方案使用相同的底座模型和相同的工具集合，差异仅体现在任务组织方式与协作机制上。

实验中，单智能体基线采用顺序执行方式，由单一智能体独立完成任务理解、代码修改和测试验证；对照方案采用多智能体但不使用共享记忆；本文方案则启用完整的层次化任务图、证据通信协议和审查—测试闭环。任务完成后，通过人工复核与自动测试结果共同评估补丁质量和任务成功情况。

表 2 仿真实验任务类别分布

任务类别	任务数量	典型内容
需求实现	16	新增接口字段、补充业务校验、实现配置开关
缺陷修复	14	空指针修复、异常处理修正、边界条件补丁
测试补全	10	新增单元测试、补充参数化测试、修正断言

4.2 评价指标与对比方案

本文采用任务完成率、缺陷修复成功率、平均迭代轮次和补丁可维护性评分四项指标进行评价。其中，任务完成率表示系统在限定轮次内达到既定完成标准的比例；缺陷修复成功率表示补丁在测试与人工复核后被判定为有效修复的比例；平均迭代轮次反映系统收敛速度；补丁可维护性评分由代码清晰度、变更范围适当性和规范一致性综合评定，满分为 5 分。

4.3 整体实验结果

从表 3 可以看出，多智能体协同框架整体优于单智能体基线。其中，本文方案的任务完成率达到 76.2%，相较于单智能体方案提高 18.7%；缺陷修复成功率达到 66.6%，相较基线提高 21.3%。这表明，通过显式任务分解和角色化协作，系统能够

更稳定地覆盖复杂开发任务中的关键步骤，减少因为任务遗漏或推理漂移导致的失败。

表 3 不同方法在仿真任务上的性能对比

方法	任务完成率	缺陷修复成功率	平均迭代轮次	补丁可维护性评分
单智能体顺序方案	64.2%	54.9%	5.3	3.3
多智能体无共享记忆	70.4%	61.5%	4.6	3.7
本文方案	76.2%	66.6%	3.9	4.1

平均迭代轮次由 5.3 轮降至 3.9 轮，说明证据驱动通信和共享记忆机制能够减少无效往返。单智能体方案中，模型经常在补丁失败后重复尝试相似修改，难以及时识别根因；而在本文方案中，测试智能体和审查智能体提供了更加具体的失败证据，使分析与编码智能体可以围绕真实问题进行定向修正，从而加快收敛。

补丁可维护性评分的提升则体现出角色分工对工程质量的价值。编码智能体专注于实现，审查智能体专注于副作用和规范一致性，测试智能体专注于验证闭环，这种分工更接近真实软件团队的协作方式，因此产生的补丁在可读性、改动范围控制和后续维护成本方面表现更好。

4.4 消融实验与案例分析

表 4 关键机制消融实验结果

方案	任务完成率	缺陷修复成功率	平均迭代轮次
完整方案	76.2%	66.6%	3.9
去除共享记忆	71.3%	61.8%	4.5
去除审查智能体	69.7%	60.4%	4.4
去除测试闭环	67.8%	58.9%	4.1

消融实验显示，共享记忆、审查角色和测试闭环都对最终效果产生了显著影响。去除共享记忆后，不同智能体容易重复定位同一问题，导致任务完成率下降；去除审查智能体后，部分补丁虽然能够通过局部测试，但在人工复核中暴露出命名不一致或异常路径缺失等问题；去除测试闭环后，系统则更容易提前结束在看似合理却未充分验证的补丁上。

在一个参数校验缺陷案例中，单智能体方案最初将问题归因于数据库字段为空，并尝试修改持久层逻辑，但测试仍然失败。本文方案中，分析智能体根据调用栈与失败断言识别出真正问题位于接口层的边界检查缺失；编码智能体据此生成局部补丁；审查智能体进一步指出异常信息不符合既有规范；测试智能体在补充参数化测试后确认补丁有效。该案例说明，多智能体协作在复杂问题上更容易形成逐步逼近的修复路径。

5 讨论与有效性威胁

本文研究仍存在若干局限。首先，实验数据集为仿真构造，虽然涵盖了常见的软件工程任务，但与真实工业级代码仓库相比，任务规模和组织复杂度仍然有限。因此，当前结论更适用于说明方法可行性，而非直接替代大规模真实项目中的严格评测。

其次，多智能体系统的效果与底座模型能力、工具调用权限和提示语设计密切相关。本文通过统一底座模型控制变量，但在实际部署中，模型版本变化、仓库结构差异和测试基础设施质量都可能影响结果。未来应结合更丰富的代码仓库和任务来源，构建标准化评测基准，以增强研究的外部效度。

最后，多智能体协同虽然提升了质量，但也引入了新的工程成本，如角色提示设计、记忆管理和消息调度复杂度。如何在收益与成本之间取得平衡，是多智能体软件工程系统从原型走向落地时必须面对的问题。后续可探索自适应角色裁剪、增量记忆压缩和安全约束控制等方向。

6 结论

本文针对软件开发任务中任务拆分困难、单智能体上下文负载过高以及缺陷修复验证不足等问题，提出了一种基于多智能体协同的软件开发任务分解与缺陷修复方法。该方法通过层次化任务图、证据驱动通信协议、审查—测试闭环与共享记忆机制，将复杂软件工程任务分配给职责清晰的多个智能体协同完成。

仿真实验结果表明，本文方法在任务完成率、缺陷修复成功率、迭代轮次和补丁可维护性等指标上均优于单智能体基线。研究结论说明，多智能体技术在软件工程领域具有较好的应用潜力，但其效果高度依赖协作机制设计。未来可进一步结合真实开源仓库、长期项目记忆和安全约束机制，推动多智能体软件开发系统向更高可靠性和更强工程实用性发展。

参考文献

- [1] Wooldridge M J. An Introduction to MultiAgent Systems[M]. 2nd ed. Hoboken: John Wiley & Sons, 2009.
- [2] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems 30 (NeurIPS 2017). 2017.
- [3] Brown T B, Mann B, Ryder N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems 33 (NeurIPS 2020). 2020.
- [4] Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can language models resolve real-world GitHub issues?[C]//The Twelfth International Conference on Learning Representations (ICLR). 2024.
- [5] Fan Z, Gao X, Mirchev M, Roychoudhury A, Tan S H. Automated repair of programs from large language models[C]//Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE). 2023.
- [6] Xia C S, Wei Y, Zhang L. Automated program repair in the era of large pre-trained language models[C]//Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE). 2023: 1482-1494. DOI: 10.1109/ICSE48619.2023.00129.
- [7] Wang X, Li B, Song Y, et al. OpenHands: An open platform for AI software developers as generalist agents[C]//The Thirteenth International Conference on Learning Representations (ICLR). 2025.
- [8] Antoniadou A, Orwall A, Zhang K, et al. SWE-Search: Enhancing software agents with Monte Carlo tree search and iterative refinement[C]//The Thirteenth International Conference on Learning Representations (ICLR). 2025.
- [9] Zhang K, Yao W, Liu Z, et al. Diversity Empowers Intelligence: Integrating expertise of software engineering agents[C]//The Thirteenth International Conference on Learning Representations (ICLR). 2025.
- [10] Le Goues C, Holtschulte N, Smith E K, et al. The ManyBugs and IntroClass benchmarks for automated repair of C programs[J]. IEEE Transactions on Software Engineering, 2015, 41(12): 1236-1256. DOI: 10.1109/TSE.2015.2454513.
- [11] Monperrus M. Automatic software repair: A bibliography[J]. ACM Computing Surveys, 2018, 51(1): Article 17. DOI: 10.1145/3105906.